Lab 3: Ring Topology & Wormhole Flow Control

王立明
2024011338

Abstract

Two Garnet extensions. Task 1: a 16-node bidirectional Ring topology (`configs/topologies/Ring.py`) with shortest-direction custom routing (`--topology=Ring --routing-algorithm=2`); the measured mean hop count of 4.0 under uniform random matches theory exactly. Task 2: a wormhole flow-control mode (`--wormhole`) letting one ctrl VC queue up to 16 single-flit packets. In the required three-way comparison, the baseline VC=1/depth=1 saturates at an accepted 0.069 packets/node/cycle on the mesh and genuinely *deadlocks* on the ring — every wedged run ends with exactly 32 in-flight packets, one per directional input VC — while VC=16×depth-1 and wormhole VC=1×depth-16, the same 16 slots per port in different shapes, are numerically indistinguishable and track the ideal accepted rate to the end of the sweep.

1 Deliverables

Table 1: Modified / added files (= contents of `Lab3_SourceCode.tar.gz`).

File	Change
`configs/topologies/Ring.py`	**new**: 1D-torus topology
`garnet/RoutingUnit.cc`	Task 1: ring routing in `outportComputeCustom()`
`configs/network/Network.py`	Task 2: `--wormhole`; sets ctrl-VC depth 16
`garnet/GarnetNetwork.py/.hh/.cc`	Task 2: `wormhole` param, `isWormholeVnet()` (plus the Lab-1 statistics changes)
`garnet/InputUnit.cc`	Task 2: several packets per VC; route carried in the flit
`garnet/SwitchAllocator.cc`	Task 2: arbitration and `send_allowed` under wormhole
`garnet/OutputUnit.cc/.hh`	Task 2: credit-based VC selection
`garnet/NetworkInterface.cc`	Task 2: injection into a busy VC

2 Task 1: Ring topology and routing

2.1 Implementation

`Ring.py` builds N = `--num-cpus` routers (16 here; the code requires $N > 2$) following the structure of `Mesh_XY.py`: one external link per attached controller, and two directed internal links per router pair — router i reaches $(i{+}1)$ mod N through its "East" port and $(i{-}1)$ mod N through "West". The custom routing (`RoutingUnit::outportComputeCustom()`, dispatched for `--routing-algorithm=2`) sends each packet along the shorter direction:

```
int cw_hops  = (dest_id - my_id + num_routers) % num_routers;
int ccw_hops = num_routers - cw_hops;
if (cw_hops <= ccw_hops) {
    assert(inport_dirn == "Local" || inport_dirn == "West");
    outport_dirn = "East";       // clockwise
} else {
    assert(inport_dirn == "Local" || inport_dirn == "East");
    outport_dirn = "West";       // counter-clockwise
}
```

Ties (antipodal destinations, $N/2$ hops either way) break towards East, so a flow always follows one path. The direction is a pure function of (router, destination), so a packet never reverses; the asserts encode that invariant and stayed silent in all runs. The routing is minimal and deterministic, but — unlike XY on a mesh — each ring direction is a cyclic channel dependency, so deadlock freedom is decided entirely by the flow control (Section 3.3).

2.2 Validation and sweep

At `--injectionrate=0.05`, received = injected and `average_hops` = 3.95; the uniform-random expectation over 16 destinations (self included) is $\big(2(1+\cdots+7)+8\big)/16 = 4.0$, confirming shortest-direction behaviour.

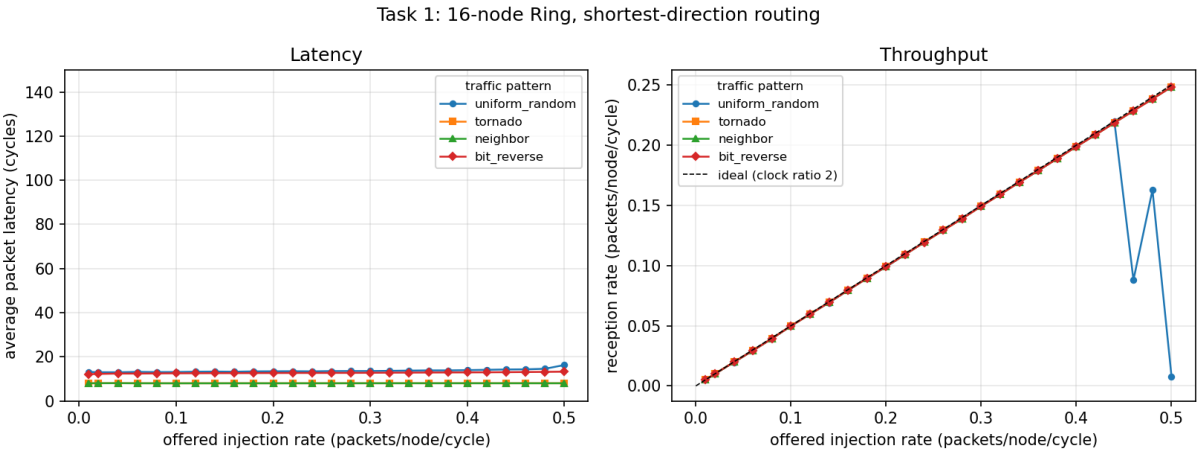

Figure 1: 16-node Ring, default router (4 VCs, 1-flit ctrl buffers). Ideal accepted rate is offered/2 (1 GHz generators vs 2 GHz network).

Table 2: Ring sweep summary.

pattern	hops	T_0	sat. (offered)	peak
uniform_random	4.07	13.2	≈ 0.46	0.220
bit_reverse	3.54	12.1	> 0.5	0.248
tornado	1.50	8.0	> 0.5	0.249
neighbor	1.50	8.0	> 0.5	0.249

$T_0 \approx 2h+5$ again — topology changes h, not the per-hop cost. Two ring-specific observations. **Tornado = neighbor**: the generator computes both on a logical 4×4 grid, where tornado's offset $\lceil 4/2 \rceil - 1 = 1$ equals neighbor's; both map to mean distance 1.5 on the ring, so no stock pattern stresses one ring direction. **Uniform random sags at ≈ 0.46 offered** although the mean link load is only $\approx 46\,\%$ of capacity there: the limit is not wire bandwidth but the baseline flow control — 4 VCs $\times$ 1-flit buffers per port, one packet per VC per credit round trip. Task 2 confirms this: with 16 slots per port the same ring tracks the ideal line to the end of the sweep.

3 Task 2: Wormhole flow control

3.1 What changes

In stock Garnet a VC is a per-*packet* resource: allocated by the head flit, held to the tail, freed with a "VC now idle" credit. On ctrl vnets (single-flit packets, 1-flit buffers) that means one packet per VC per credit round trip. `--wormhole` removes this coupling for the ctrl vnets (the

only ones carrying the lab's single-flit `HEAD_TAIL_` packets): the ctrl buffer depth becomes 16 (`buffers_per_ctrl_vc = 16`, enforced by the existing `decrement/increment_credit` machinery), and up to 16 packets queue in one VC. All changes are guarded by one helper, so data vnets and non-wormhole runs keep stock behaviour:

```
bool isWormholeVnet(int vnet) const
{ return m_wormhole && m_vnet_type[vnet] == CTRL_VNET_; }
```

The four functional changes:

1. **InputUnit::wakeup()** — a HEAD_TAIL_ flit may find its VC ACTIVE_ (earlier packets still queued); since queued packets can head for different outports, the route is computed per packet and stored *in the flit*, not in the VC.

2. **SwitchAllocator** — SA-I and the `send_allowed` ordering check read the outport of the packet at the head of the VC (`peekTopFlit`); `send_allowed` also accepts an ACTIVE_ output VC that still has credits.

3. **OutputUnit / vc_allocate** — `select_free_vc(vnet, allow_active)` may pick an active VC with spare credits: the packet queues behind the previous one downstream.

4. **SA-II, after a HEAD_TAIL_ wins** — if more packets wait in the input VC it stays ACTIVE_, its granted outvc is cleared so the next packet re-allocates, and a *plain* credit is returned; the "VC idle" credit is sent only when the buffer drains. **NetworkInterface::calculateVC()** applies the same idle-or-credited rule at injection.

Correctness: without `--wormhole` the Lab-1 reference run reproduces its numbers exactly (3165/3162, latency 15.557559); `config.ini` of a wormhole run shows `wormhole=true`, `buffers_per_ctrl_vc=16` packet conservation holds in all 234 sweep runs.

3.2 Three-way comparison

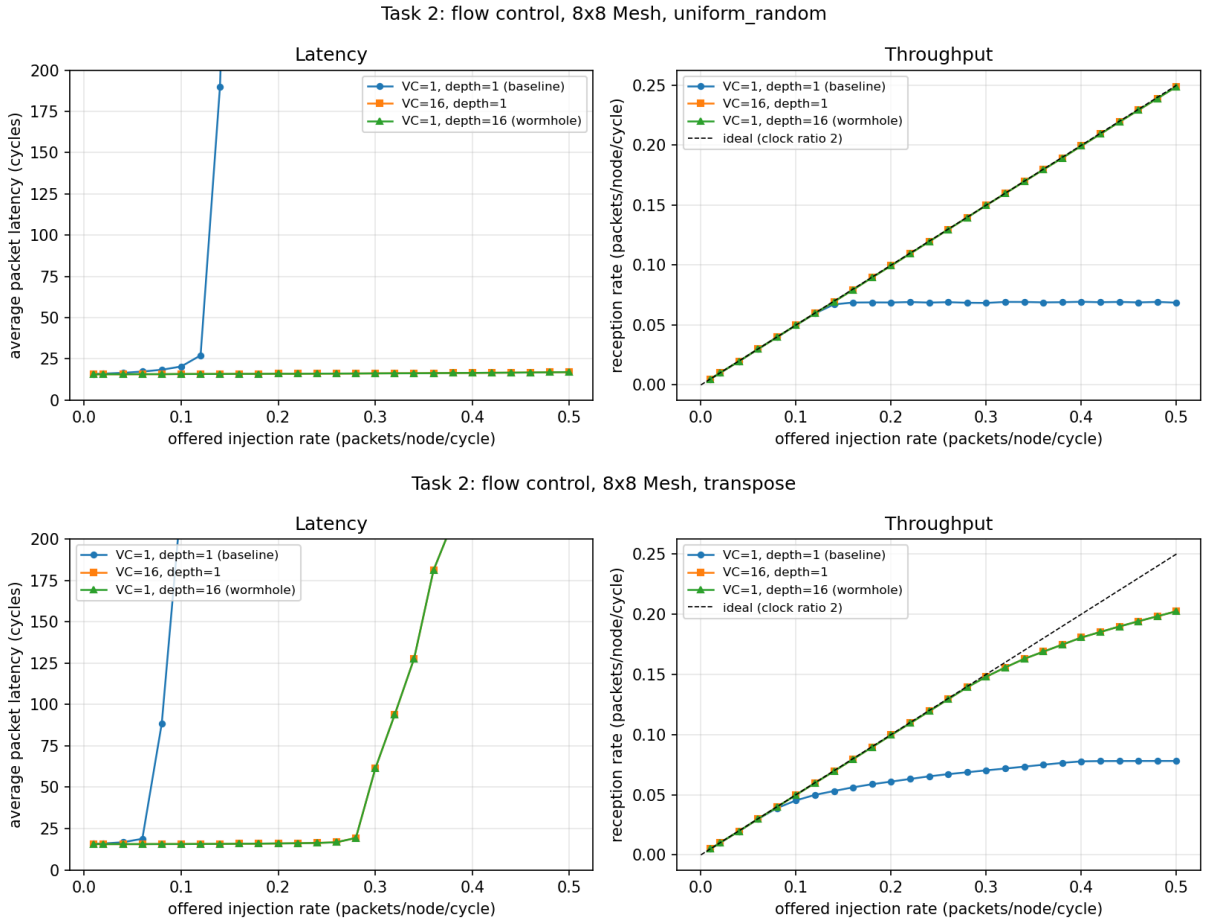

Figure 2: 8×8 Mesh: uniform random (top), transpose (bottom). The VC=16 and wormhole curves coincide.

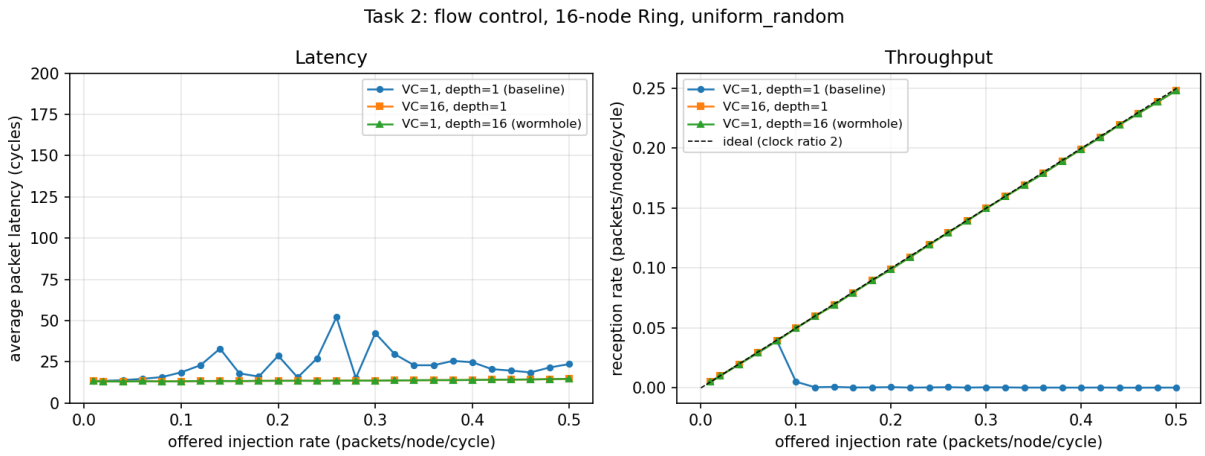

Figure 3: Same comparison on the 16-node Ring. The baseline collapses to near-zero delivered traffic beyond offered 0.08: deadlock.

Table 3: Summary (T_0 zero-load latency; sat. = accepted rate departs ideal; peak = highest accepted rate).

network / pattern	config	T_0	sat.	peak
Mesh / uniform	VC=1, depth=1	15.7	0.16	0.069
Mesh / uniform	VC=16, depth=1	15.6	>0.5	0.249
Mesh / uniform	VC=1, depth=16 (wormhole)	15.6	>0.5	0.249
Mesh / transpose	VC=1, depth=1	15.8	0.10	0.078
Mesh / transpose	VC=16, depth=1	15.7	0.36	0.203
Mesh / transpose	VC=1, depth=16 (wormhole)	15.7	0.36	0.203
Ring / uniform	VC=1, depth=1	13.4	0.10 (deadlock)	0.039
Ring / uniform	VC=16, depth=1	13.2	>0.5	0.248
Ring / uniform	VC=1, depth=16 (wormhole)	13.2	>0.5	0.248

VC=1, depth=1 collapses early. One packet slot per port, reusable only after the full credit round trip ($\approx$ 5 cycles), caps every link far below one packet per cycle and couples unrelated flows head-on. On the mesh the accepted rate flattens at 0.069/0.078; past the knee latency is pure source queueing. On the ring it is worse — outright deadlock (Section 3.3).

16 slots per port remove the bottleneck — in either shape. VC=16×depth-1 and wormhole VC=1×depth-16 offer the same 16 packet slots and produce numerically indistinguishable curves (mesh/uniform at 0.5: accepted 0.2489 both, latency 16.85 vs 16.86). For single-flit packets the mechanisms converge: SA picks at most one flit per input port per cycle anyway, so 16 parallel shallow queues buy nothing over one deep FIFO. The residual differences are architectural: wormhole needs one VC's state (simpler allocation) but is exposed to head-of-line blocking between packets for different outports; VC=16 pays 16 allocator candidates per port.

What buffering cannot do. Mesh/transpose still flattens at 0.203 with 16 slots — identical to Lab 2's VC=4 result. That ceiling is the capacity of the XY turn links, a topological bottleneck: flow control decides how close to link capacity a network operates, it cannot create capacity.

3.3 Deadlock on the ring

XY on the mesh has an acyclic channel-dependency graph, so all three mesh configurations merely saturate. Each ring direction, however, is a dependency cycle, and the sweep shows both outcomes:

- **VC=1, depth=1 deadlocks.** From offered 0.10 upwards only ~70–140 packets are injected in 10000 cycles (vs. 6300 at 0.08) and almost none delivered. The smoking gun: *every* wedged run ends with exactly 32 packets in flight = 16 routers × 2 directions × 1 VC — each directional input VC holds one packet waiting for the credit of the next router in the cycle; none can move, and the NIs stall on the single VC. (The run ends because `sim-cycles` expires; the built-in panic fires only at 50000 cycles.)
- **Wormhole (depth 16) does not deadlock here.** Deadlock needs every buffer around the cycle full of packets that continue within it. At effective loads ≤ 0.25 packets/node/cycle — below the ring's 0.5 capacity — queues stay short and the cycle never closes. This is an operating-point guarantee, not structural: driven past saturation long enough, single-VC wormhole on a ring reaches the same all-full state. **VC=16** avoids it likewise, with the extra margin that a blocked packet leaves 15 VCs usable.

A production ring makes deadlock impossible by construction with a *dateline*: two VC classes, packets switch class when crossing a fixed link, breaking the cycle. Since this implementation carries per-packet routing in the flit, a dateline would only touch `calculateVC()`/`select_free_vc()` — the VCs to host it exist whenever `vcs_per_vnet` ≥ 2.

4 Conclusion

The Ring behaves exactly as first-order theory predicts — hop counts, $T_0 \approx 2h + 5$, and the degenerate tornado/neighbor mapping are all reproduced. The wormhole mode removes the baseline's packet-per-VC restriction, and the three-way comparison isolates its cost: one slot per port runs the mesh at a fraction of link capacity and deadlocks the ring outright, while 16 slots — whether 16 shallow VCs or one deep wormhole FIFO — recover the full ideal throughput for single-flit traffic. Flow control decides how much of a topology's capacity is usable, and on a cyclic topology whether it is usable at all; robustness past saturation would additionally require a dateline VC scheme.